

A Technical Report on Laptop Price Analysis Using Machine Learning
*Analyzing laptop prices based on brand, operating system, GPU, screen resolution, memory,
and CPU using Multiple Linear Regression, Decision Tree Regression, and Random Forest
Regression*

Machine Learning Models

By

Ibialabo Dappa
ibialabodappa@gmail.com

Dated
16 July 2026

Abstract

This report presents machine learning techniques for predicting and analyzing laptop prices using features such as manufacturer, operating system, graphics processing unit (GPU), screen resolution, memory capacity, and central processing unit (CPU). The goal is to predict laptop prices across different manufacturers and to understand why some laptops are more expensive than others based on their specifications and other relevant attributes, using Multiple Linear Regression, Decision Tree Regression, and Random Forest Regression models. A dataset containing columns such as company, type name, screen size (in inches), screen resolution, CPU, RAM, memory, GPU, operating system, weight, and price was used, and feature engineering was performed on several categorical variables. From the screen resolution, features such as screen width, height, touchscreen indicator, and IPS panel indicator (in binary form) were extracted; from the CPU, the brand name and clock speed were derived; from the GPU, only the GPU brand name was retained; and from the memory column, separate features for SSD, HDD, flash storage, and hybrid storage were created. The original columns corresponding to these engineered features were then dropped. All categorical variables were encoded into binary or numerical form using appropriate machine learning-oriented encoding techniques. The Multiple Linear Regression, Decision Tree Regression, and Random Forest Regression models were trained on 80% of the processed dataset and achieved strong performance in predicting laptop prices. The models were subsequently evaluated on the remaining 20% test set, producing final predictions that included both the original laptop prices and their corresponding predicted values. This report outlines the entire workflow from data preprocessing to model evaluation, highlighting challenges such as feature engineering, categorical encoding, and ensuring robust predictive performance. The final models demonstrated practical utility for real-world use and can be applied by companies, as well as small and medium-sized laptop retailers, to support data-driven pricing and smarter business decisions.

1. Introduction and Background

Laptops are now essential for education, business, entertainment, and software development, with diverse configurations and price ranges. The explosion of models, frequent product updates, and varying vendor strategies make it hard for consumers and retailers to determine fair prices based on specifications like CPU, memory, storage, graphics, and display. For consumers, understanding how hardware features affect price is key to informed buying, while for manufacturers and retailers, data-driven insight supports competitive pricing, inventory planning, and targeted marketing. Traditional, manual pricing methods often struggle to capture complex, non-linear relationships among multiple laptop specifications..

Machine learning offers a robust framework for modeling these complex relationships by learning patterns from historical laptop data and leveraging them to predict prices for new or existing configurations. In this project, supervised regression models are used to analyze how factors such as brand (company), type name, screen size (inches), screen resolution, CPU characteristics, RAM, storage configuration (SSD, HDD, flash, hybrid), GPU brand, operating system, and weight contribute to overall laptop pricing. Multiple Linear Regression, Decision Tree Regression, and Random Forest Regression are employed to capture both linear and non-linear dependencies between these features and the target variable (price).

A key challenge in this project is that the dataset's detailed laptop specifications required substantial feature engineering before modeling. Screen resolution was decomposed into width, height, touchscreen, and IPS indicators; CPU features such as brand and clock speed were extracted; GPU information was reduced to GPU brand; and storage was split into SSD, HDD, flash, and hybrid components, with redundant columns removed and categorical variables numerically encoded for use in the models. This engineered dataset, combined with Multiple Linear Regression, Decision Tree Regression, and Random Forest Regression, forms the core methodology of the project, with all resources and materials duly cited in the references section

.

2. Dataset and Methods

The dataset consists of laptop records with fields such as brand (company), type name, screen size (inches), screen resolution, CPU, RAM, memory, GPU, operating system, weight, and price. The data was already cleaned prior to modeling, containing no missing values, nulls, or duplicates. Feature engineering was then applied by decomposing the screen resolution into width, height, touchscreen indicator, and IPS panel indicator, extracting CPU brand and laptop speed, deriving GPU brand, and splitting the memory column into separate SSD, HDD, flash storage, and hybrid storage features, after which redundant original columns were removed. The resulting processed dataset was used to train three regression models Multiple Linear Regression, Decision Tree Regression, and Random Forest Regression models .

The full implementation and source code for the project can be accessed on GitHub:

2.1 Dataset

This project made use of a dataset sourced from Kaggle, obtained in CSV format. The original dataset consisted of 1,303 rows and 12 columns, containing records of laptop models and their key technical specifications. Following feature engineering — including the derivation of new variables and one-hot encoding of categorical fields — the dataset was split into training and test sets, then further separated into input (X) and target (y) subsets. The resulting X_train dataset contains 1,042 rows and 57 columns and the The corresponding target subset, y_train, contains 1,042 rows and 1 column which is the price

The following are the column names and their descriptions, grouped by category

- Numeric specifications
- Inches: Screen size of the laptop, measured in inches.
- Ram: Amount of installed RAM, measured in GB.
- Weight: Weight of the laptop, measured in kilograms.
- height / width: Screen resolution dimensions (pixel height and width), derived from the original ScreenResolution field.
- speed: CPU clock speed, in GHz, extracted from the original CPU field.

- SSD, HDD, flash_storage, Hybrid: Storage capacity (in GB) for each respective storage type, derived from the original Memory field.
- IPS: Binary indicator (1/0) denoting whether the display uses IPS panel technology.
- Touchscreen: Binary indicator (1/0) denoting whether the laptop has a touchscreen.
- Price: The target variable representing the price of the laptop (the value the model is trained to predict).

- Categorical encoded columns
- Company (one-hot encoded)

Company_Acer, Company_Apple, Company_Asus, Company_Chuiwi, Company_Dell, Company_Fujitsu, Company_Google, Company_HP, Company_Huawei, Company_LG, Company_Lenovo, Company_MSI, Company_Mediacom, Company_Microsoft, Company_Razer, Company_Samsung, Company_Toshiba, Company_Vero, Company_Xiaomi: Binary indicators representing the laptop manufacturer/brand.

- TypeName (one-hot encoded)

TypeName_2in1 Convertible, TypeName_Gaming, TypeName_Netbook, TypeName_Notebook, TypeName_Ultrabook, TypeName_Workstation: Binary indicators representing the laptop category.

- Operating System (one-hot encoded)

OpSys_Android, OpSys_ChromeOS, OpSys_Linux, OpSys_MacOSX, OpSys_NoOS, OpSys_Windows10, OpSys_Windows10S, OpSys_Windows7, OpSys_macOS: Binary indicators representing the installed operating system.

CPU Brand (one-hot encoded)

cpu_brand_name_AMD, cpu_brand_name_Intel Core i3, cpu_brand_name_Intel Core i5, cpu_brand_name_IntelCorei7, cpu_brand_name_Samsung, cpu_brand_name_other intel: Binary indicators representing the processor brand/series.

- GPU Brand (one-hot encoded)

Gpu_brand_AMD, Gpu_brand_ARM, Gpu_brand_Intel, Gpu_brand_Nvidia: Binary indicators representing the graphics processing unit brand.

Meanwhile, the test dataset was similarly divided into input and target subsets. The `X_test` dataset consists of 261 rows and 57 columns, containing the same predictor variables as `X_train`. The `y_test` dataset consists of 261 rows and 1 column, containing the corresponding target variable, Price. The columns in `X_test` are exactly the same as those in `X_train`, ensuring consistency between the data the model was trained on and the unseen data it is evaluated against. This allows the model, once trained on `X_train` and `y_train`, to generate price predictions on `X_test`, which can then be compared against the actual values in `y_test` to assess model performance.

2.2 Data Processing

Amongst all the other analytical methods employed in this project, the data processing stage is the most crucial, as it determines how well or how poorly the resulting model performs. It is worth noting the "80/20" rule commonly observed in data analytics, which states that approximately 80% of a data analyst's time is spent on training the dataset, while the remaining 20% goes into modeling and statistical analysis. In this phase of the project, the following steps were taken in processing the raw laptop specifications data:

- Choose relevant columns: The relevant columns for this analysis were Company, TypeName, Inches, Ram, OpSys, Weight, width, height, Touchscreen, IPS, cpu_brand_name, speed, Gpu_brand, SSD, HDD, flash_storage, Hybrid, and Price — the last of which served as the target variable.
- Data cleaning: The dataset was found to be already clean prior to modeling, containing no missing values, null entries, or duplicate records, and therefore required no additional imputation or deduplication steps.
- Exploratory Data Analysis
Prior to model development, exploratory data analysis (EDA) was conducted to understand the relationship between key categorical and numerical features and laptop price. The following visualizations were generated to support this analysis.
 - Company Distribution (Count)
In addition to examining price by manufacturer, the raw distribution of laptop records by company was also analyzed to understand the composition of the

dataset. This bar chart shows that Dell, Lenovo, and HP dominate the dataset, each accounting for close to 300 records, making them by far the most represented brands. Asus and Acer follow at a moderate level, with roughly 160 and 100 records respectively. The remaining manufacturers — including MSI, Toshiba, Apple, Samsung, Mediacom, Razer, Microsoft, Vero, Xiaomi, LG, Chuwi, Google, Fujitsu, and Huawei — each contribute relatively few records, generally fewer than 50.

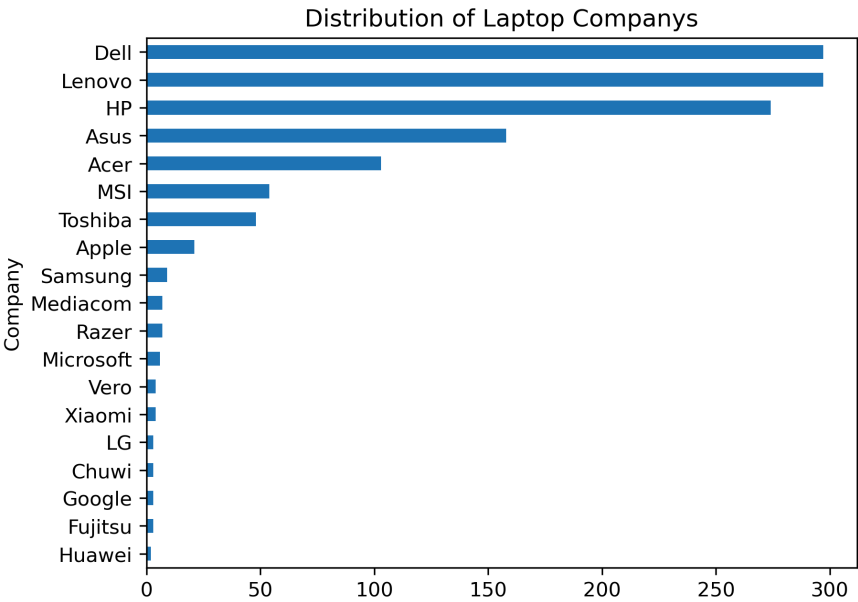


Figure 1

- Operating System vs. Price

The bar chart of average price by operating system reveals a clear pricing hierarchy across platforms. macOS and Windows 7 command the highest average prices, both exceeding ₦85,000, while Windows 10 S and Mac OS X occupy a mid-to-upper price tier. Windows 10, despite being the most widely used operating system in the dataset, sits in the middle range, suggesting that its prevalence spans both budget and premium devices. At the lower end, Linux, No OS, Chrome OS, and Android are associated with the least expensive laptops, which aligns with expectations, as these operating systems are typically found on lightweight, budget-oriented, or Chromebook-style devices.

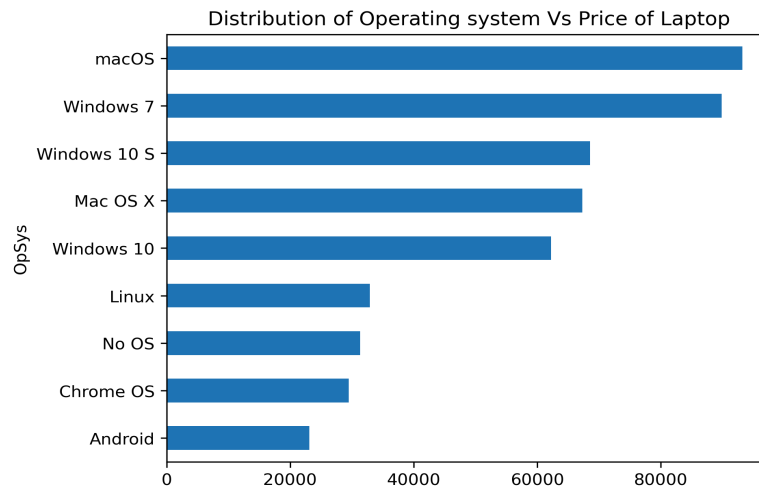


Figure 2

- Company vs. Price

The distribution of average price by manufacturer shows considerable variation across brands. Razer stands out significantly, with an average price far exceeding all other manufacturers — consistent with its market positioning as a premium gaming laptop brand. LG, MSI, Google, Microsoft, and Apple form a distinct upper-mid tier, reflecting their focus on premium or specialized devices (gaming, Chromebook Pixel-style, Surface, and MacBook lines, respectively). Mainstream manufacturers such as Dell, Asus, Lenovo, and HP cluster in the mid-range, while Acer, Chuwi, Mediacom, and Vero occupy the lowest price tier, reflecting their positioning as budget-focused

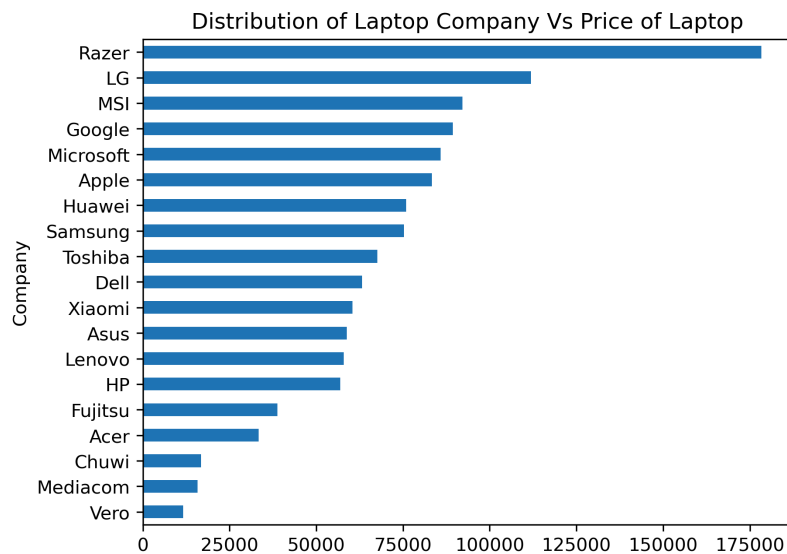


Figure 3

- **TypeName vs. Price**

Laptop category also shows a strong association with price. Workstation laptops have the highest average price, which is expected given their target use case of demanding professional and computational workloads. Gaming laptops follow closely, reflecting the premium hardware (dedicated GPUs, high-refresh displays) required for that segment. Ultrabooks and 2-in-1 Convertibles occupy the mid-range, balancing portability with performance. Notebooks and Netbooks are the least expensive categories, consistent with their design as basic, everyday-use devices.

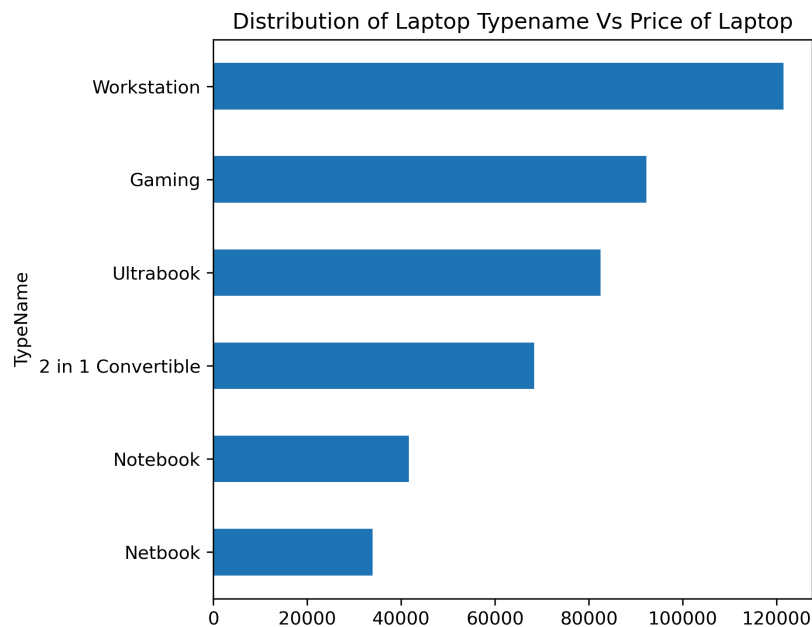


Figure 4

- **Price vs. Screen Size (Inches)**

The scatter plot of price against screen size shows a weak-to-moderate positive relationship, but with substantial spread at nearly every screen size. Laptops clustered around 13.3", 14", 15.6", and 17.3" — the most common industry-standard sizes — display prices ranging from roughly ₦10,000 at the low end to over ₦300,000 at the high end within the same screen size, indicating that screen size alone is a weak predictor of price. Notably, the highest-priced laptops in the dataset (above ₦280,000) are concentrated around the 17.3" and 14" sizes, suggesting that larger screens are more common among premium workstation and gaming models, though smaller high-end

ultrabooks (e.g., near 14") also reach high price points. The wide vertical spread at each screen size reinforces the earlier finding that other specifications — brand, type, CPU, GPU, and RAM — play a more decisive role in determining price than screen size in isolation.

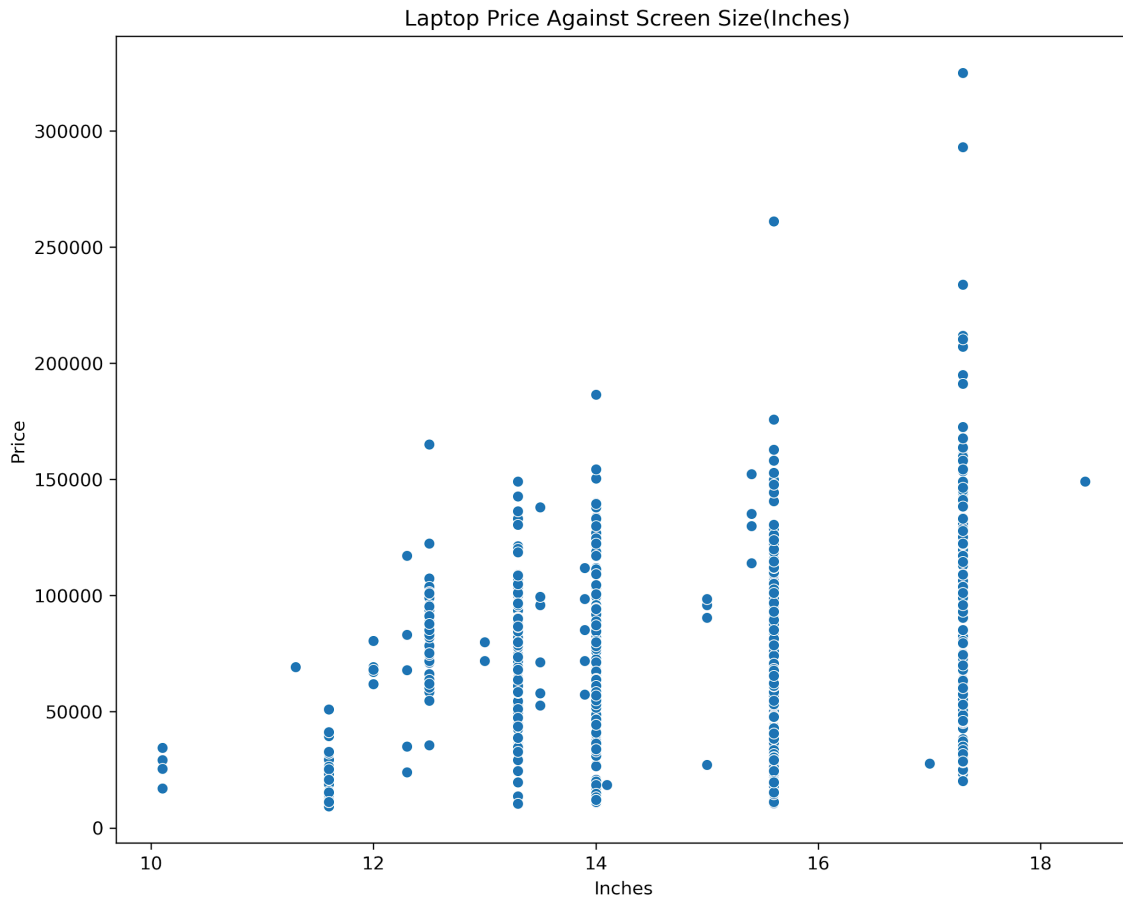


Figure 5

Collectively, these visualizations indicate that laptop price is driven more strongly by categorical factors — brand reputation, laptop category, and operating system — than by screen size alone. Premium and specialized segments (Razer, Workstation, Gaming, macOS) consistently command higher prices, while budget-oriented brands, categories, and operating systems (Vero, Netbook, Chrome OS, Android) cluster at the lower end. These patterns provided useful direction for feature selection and reinforced the importance of including categorical variables such as Company, TypeName, and OpSys in the regression models.

- Feature engineering: Several columns were split into more columns, using the feature engineering process such as:
 - ScreenResolution was broken down into width and height (pixel dimensions), along with binary indicators for Touchscreen and IPS panel presence.
 - Cpu was processed to extract the cpu_brand_name (processor brand/series) and speed (clock speed in GHz).
 - Gpu was processed to derive the Gpu_brand (graphics processing unit brand).
 - Memory was split into separate storage-type columns: SSD, HDD, flash_storage, and Hybrid, each representing capacity in GB.
 - Column removal: Once the above features were extracted, the original redundant columns — ScreenResolution, Cpu, Gpu, and Memory — were removed from the dataset, as their relevant information had already been captured in the newly engineered features listed above.

- Correlation Matrix (Post Feature Engineering)

Following feature engineering, a correlation matrix was generated across all numerical columns to examine the linear relationships between the engineered features and the target variable, Price. Several key patterns emerged:

- Ram showed the strongest positive correlation with Price (0.74), confirming it as one of the most influential predictors of laptop price in the dataset.
- SSD also displayed a strong positive correlation with Price (0.67), reflecting the premium typically associated with solid-state storage over traditional hard drives.
- width and height (screen resolution dimensions) showed moderate positive correlations with Price (0.56 and 0.55 respectively), and were almost perfectly correlated with each other (0.99), as expected given that both are derived from the same ScreenResolution field. This near-perfect correlation flags a potential multicollinearity concern, which is particularly relevant for the Multiple Linear Regression model, as highly correlated predictors can inflate coefficient variance and reduce interpretability, even though it typically has minimal impact on tree-based models.
- speed (CPU clock speed) showed a moderate positive correlation with Price (0.43), indicating that faster processors are associated with higher-priced laptops.

- HDD showed a weak negative correlation with Price (-0.095), consistent with the earlier observation that HDD-equipped laptops tend to be positioned at the budget end of the market, in contrast to SSD-equipped models.
- Inches and Weight were very strongly correlated with each other (0.83), which is intuitive, as larger screens generally require larger, heavier chassis. However, both showed only weak correlations with Price (0.068 and 0.21 respectively), reinforcing the earlier scatter plot finding that screen size alone is a weak predictor of price.
- flash_storage and Hybrid showed negligible correlation with Price, suggesting these storage types are not strong price differentiators in this dataset, likely due to their limited representation among modern laptop configurations.

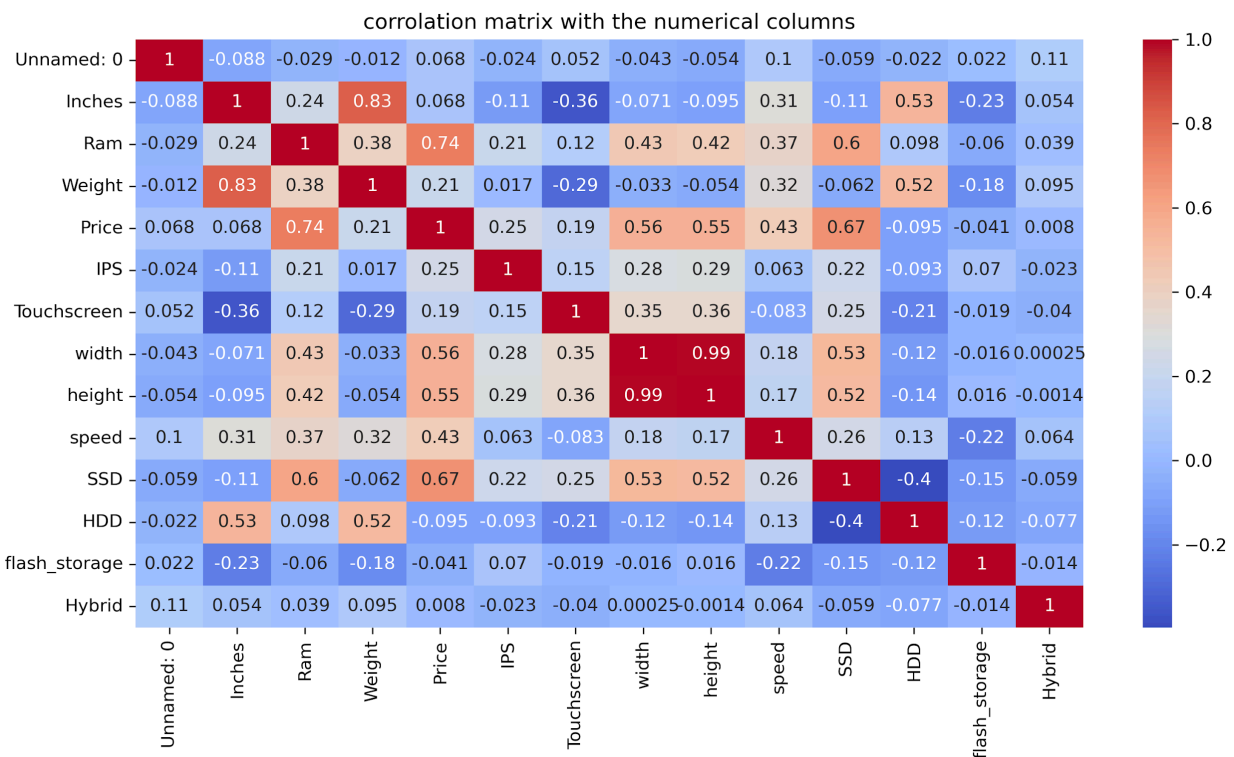


Figure 6

Summary of Correlation Findings:

The correlation analysis confirms that RAM and SSD capacity are the strongest numerical predictors of laptop price, while screen dimensions (width/height) and CPU speed contribute moderately. The high correlation between width and height, as well as

between Inches and Weight, highlights areas of multicollinearity that should be considered when interpreting the coefficients of the Multiple Linear Regression model, though this is expected to have less impact on the Decision Tree and Random Forest models given their non-parametric, split-based nature.

- One-Hot Encoding: This step was applied to categorical variables such as Company, TypeName, OpSys, cpu_brand_name, and Gpu_brand, converting each unique category into a separate binary column (e.g., Company_Dell, Company_HP), where a value of 1 indicates the presence of that category and 0 indicates its absence. This technique was used because these variables had no inherent ordinal relationship between categories, and one-hot encoding allowed the model to treat each brand, type, and operating system as an independent, non-hierarchical feature — preventing the model from incorrectly assuming a numerical ranking between, for instance, Dell and HP.
- Label Encoding was applied consistently across all categorical variables in the dataset to transform them into a machine-readable numerical format. Each unique category within a column (e.g., each laptop brand under Company, or each operating system under OpSys) was assigned a distinct integer label, allowing the model to process these variables without requiring the creation of additional binary columns. This approach simplified the feature set and kept the dimensionality of the training data manageable, in contrast to one-hot encoding, which would have significantly expanded the number of columns by creating a separate binary indicator for every unique category.
- Both encoding techniques were used deliberately for different types of variables in order to produce optimal results.

2.3 Machine Learning Methods

This project implemented and iteratively refined three regression algorithms to predict laptop prices from hardware specifications, brand, and design characteristics: Linear Regression, Decision Tree Regressor, and Random Forest Regressor. Model development followed an iterative methodology in which an initial baseline was established, diagnosed for weaknesses, and progressively refined through targeted interventions — including categorical encoding comparison, multicollinearity reduction, and target variable transformation. This section documents the complete modeling process, from initial implementation through to the final, validated results.

All models were implemented using Scikit-learn's Pipeline framework, which chained preprocessing and estimation steps together to ensure consistent, leakage-free transformations between training and test data.

Baseline Model Development

- **Feature Selection and Encoding Strategy**

Eighteen features — including *Company, TypeName, Inches, Ram, OpSys, Weight, width, height, Touchscreen, IPS, cpu_brand_name, speed, Gpu_brand, SSD, HDD, flash_storage and Hybrid— were retained as predictors, with Price as the target variable. The dataset was confirmed to be free of missing values and duplicate records prior to modeling, requiring no additional cleaning at this stage.

To evaluate the influence of categorical feature representation on model performance, two parallel versions of the dataset were constructed:

- One-Hot Encoding, applied to Company, TypeName, OpSys, cpu_brand_name, and Gpu_brand, converting each category into an independent binary column and preserving the non-ordinal nature of these variables.
- Label Encoding, which assigned each category a distinct integer value, producing a more compact feature representation at the risk of implying a false ordinal relationship between unrelated categories

Each of the three models was trained and evaluated on both dataset variants under identical train-test split conditions, allowing the effect of encoding strategy to be isolated from the effect of model choice.

Baseline Results

The initial models, trained on the untransformed target variable, produced the following results:

One-Hot Encoded Dataset

Model	MAE	RMSE	R ² Score
Linear Regression	12,429.52	17,902.61	0.7777
Decision Tree Regressor	13,079.62	19,913.70	0.7250
Random Forest Regressor	10,243.74	16,673.12	0.8072

Label Encoded Dataset

Model	MAE	RMSE	R ² Score
Linear Regression	14,167.69	-	0.7209
Decision Tree Regressor	13,015.37	-	0.7421
Random Forest Regressor	10,131.59	-	0.8051

At this baseline stage, Random Forest was the strongest performer under both encoding strategies, while Linear Regression showed the largest sensitivity to encoding choice — achieving a meaningfully higher R² under one-hot encoding (0.778) than under label encoding (0.721). This pattern is consistent with the theoretical expectation that linear models are more

susceptible to the false ordinal relationships that label encoding can introduce between unrelated categories, whereas tree-based models, which split on value thresholds rather than assuming linear relationships, are comparatively robust to this limitation.

Having established this baseline, the project proceeded to identify and address the specific weaknesses limiting Linear Regression's performance, given its comparatively weaker showing relative to Random Forest.

Model Improvement Process

Improving a model's performance is rarely a single action; rather, it involves systematically testing hypotheses about why a model underperforms and validating each proposed fix against the data before adopting it. The following diagnostic and refinement steps were undertaken in sequence.

- **Step 1: Addressing Multicollinearity via Dummy Variable Encoding**

One-hot encoding is known to introduce a structural form of multicollinearity, since the binary columns generated for each categorical group are linearly dependent (their values necessarily sum to 1 for every record). As a first intervention, the encoder was configured to drop one reference category per categorical group (`drop="first"`), a standard technique for mitigating the "dummy variable trap."

This adjustment was tested empirically and found to produce no meaningful change in Linear Regression's R^2 score. This outcome, while initially unexpected, is consistent with the underlying numerical properties of Scikit-learn's least-squares solver, which handles perfectly collinear columns without producing invalid results — the redundancy affects the stability and interpretability of individual coefficients rather than the model's overall predictive accuracy on the test set. This step was therefore retained for coefficient interpretability but was not treated as a performance-improving intervention in its own right.

- **Step 2: Multicollinearity Reduction Among Numeric Features**

A correlation analysis of the numeric feature set revealed a strong linear relationship between the width and height variables (representing screen resolution dimensions), an expected finding given that higher-resolution displays tend to scale proportionally along

both axes. To reduce redundancy in the feature set, the height variable was removed, retaining width as a representative proxy for display resolution. This adjustment was confirmed not to reduce the model's explanatory power, supporting the conclusion that height contributed no meaningful independent signal beyond what width already captured.

- **Step 3: Target Variable Transformation**

Diagnostic inspection of the dataset's price distribution revealed pronounced right-skewness, driven by a comparatively small number of high-priced premium and gaming laptop models. Because Linear Regression assumes approximately linear, homoscedastic relationships between features and target, this skew was identified as a likely source of the model's comparatively weaker fit.

To address this, a natural logarithmic transformation, $\log(1 + \text{Price})$, was applied to the target variable prior to model training, with predictions subsequently subject to comparable inverse transformation where required for interpretation. This intervention produced the most substantial improvement observed during the refinement process, increasing Linear Regression's R^2 score from 0.778 to 0.839 — a gain of approximately six percentage points — confirming that the untransformed model's fit had indeed been constrained by the skewed distribution of the target variable

Final Model Results

Following the completion of this refinement process, the final performance of all three models — trained on the log-transformed target variable using the one-hot encoded dataset — is presented below.

Model	MAE(Log Scale)	RMSE(Log Scale)	R ² Score
Linear Regression	0.189	0.239	0.8387
Decision Tree Regressor	0.221	0.304	0.7388
Random Forest Regressor	0.166	0.222	0.8604

Note on error metric interpretation: As a consequence of the log transformation applied to the target variable, the MAE and RMSE values reported above are expressed in log-transformed units rather than the original currency of the dataset, and are therefore not directly comparable in magnitude to the untransformed baseline results presented earlier in this section. The R^2 score, by contrast, is a scale-invariant measure and remains directly comparable across both the transformed and untransformed model runs, providing a consistent basis for evaluating the overall improvement achieved through this refinement process.

Findings and Insights

- **Random Forest remained the strongest-performing model throughout the refinement process, achieving the highest R^2 score (0.860)** following the log transformation, and consistently outperforming both alternative models across every stage of development. This confirms the suitability of ensemble tree-based methods for this prediction task, owing to their capacity to model non-linear feature interactions while resisting the overfitting tendencies observed in the single Decision Tree.
- **Linear Regression benefited most substantially from the log transformation**, narrowing the performance gap with Random Forest considerably (from a twelve-percentage-point difference at baseline to roughly two percentage points following transformation). This finding reinforces the theoretical expectation that linear models are particularly sensitive to skewed target distributions, and that addressing this skew can meaningfully improve their competitiveness relative to non-parametric alternatives.
- **The Decision Tree Regressor underperformed relative to both other models following the target transformation**, a result attributable to its known tendency to overfit training data in the absence of depth constraints or pruning — a limitation that Random Forest's ensembling approach is specifically designed to address.
- **Practical implication.** Given its consistently superior performance across baseline and refined evaluations alike, the Random Forest Regressor, trained on log-transformed price data using one-hot encoded categorical features, is recommended as the final

model for this laptop price prediction task. Linear Regression, following refinement, represents a viable and considerably more interpretable alternative where model transparency is prioritized over marginal gains in predictive accuracy.

References

- Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.
<https://doi.org/10.1023/A:1010933404324>
- Scikit-learn Developers. RandomForestRegressor — Scikit-learn Documentation
<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html>
- Potdar, K., Pardawala, T. S., & Pai, C. D. (2017). A Comparative Study of Categorical Variable Encoding Techniques for Neural Network Classifiers. International Journal of Computer Applications, 175(4), 7–9.
<https://doi.org/10.5120/ijca2017915495>

